
ASG2 Introduction

Secure Software & Hardware Systems

Background

“After showing your skills in security via automation, fuzzing, and vulnerability reporting, EvilCorp has a new task for you: Help them to recover key information about their legacy hardware security token, hosted on the Raspberry Pi Pico!

Unfortunately, the engineer who originally created the token left the company and documentation is missing. You will need to probe the different communication interface and recover the Pin code to unlock the device. “

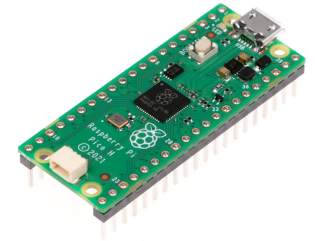
– <https://canvas.bham.ac.uk/courses/83848/assignments/554063>

The Hardware

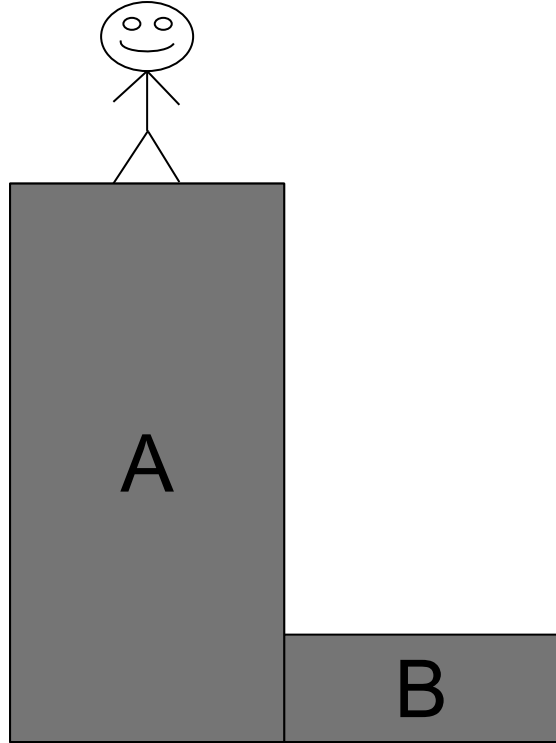
Bill of Materials:

- 1x Logic Analyzer kit, including:
 - Jumper Wires
 - USB A<->Mini B

- 1x Raspberry Pi Pico kit, including:
 - USB A <-> Micro



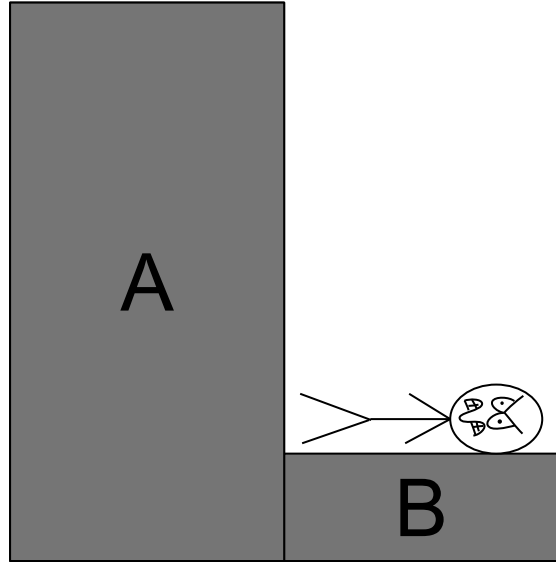
Warning: short circuit



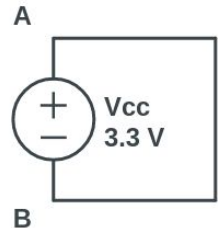
- Stickman is standing on A.
- This scenario is called open circuit. No current is flowing so it is intrinsically safe.



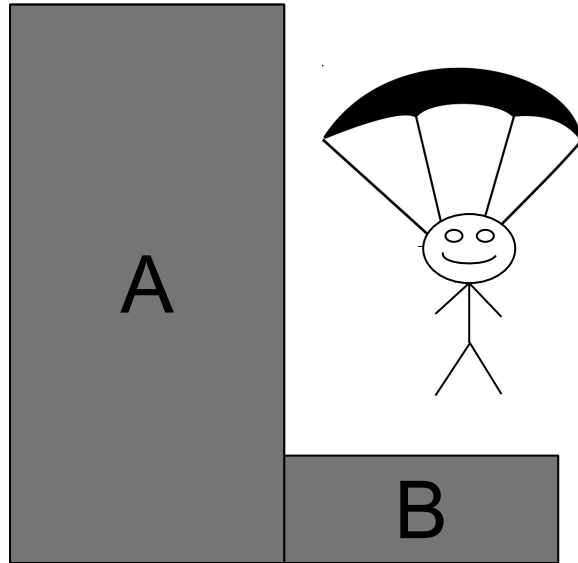
Warning: short circuit



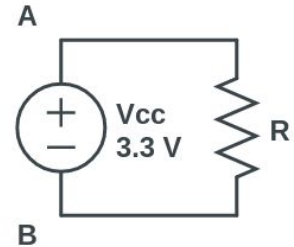
- Stickman takes a step towards B. The air resistance is low, so we hit the ground at a high speed.
- This is called short-circuit and the high current may damage components.



Warning: short circuit



- Stickman decides to use a parachute to increase his air resistance.
- If the resistance is big enough, the current will not damage any component.

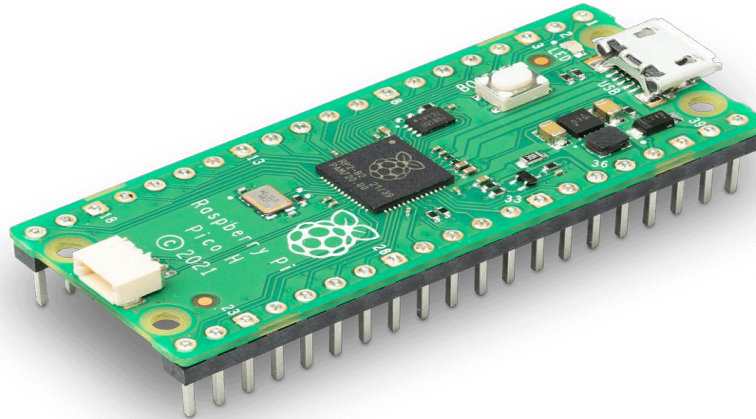


Warning: short circuit

- **NEVER** connect pins at 3.3V or 5V to ground. This will cause a so called short-circuit and it can damage your device permanently.
- Establish common ground connection between components in your circuit
 - In this exercise: connect ground of logic analyzer to ground of pico when used together
 - Will create a unified reference

The Pico Board

Raspberry-Pi Pico board ([datasheet](#)).



The Pico Board

Raspberry-Pi Pico board ([datasheet](#)).



RP2040 A microcontroller by Raspberry Pi

RP2040 Datasheet

A microcontroller
by Raspberry Pi

Assignments

- Investigate the hardware target
 - Flash the target firmware
 - Find and decode a UART message
 - Find and decode a SPI message

} **Assignment 2A**

- Carry out function level rehosting
 - Recover lost pin code

} **Assignment 2B**

Provided Binaries

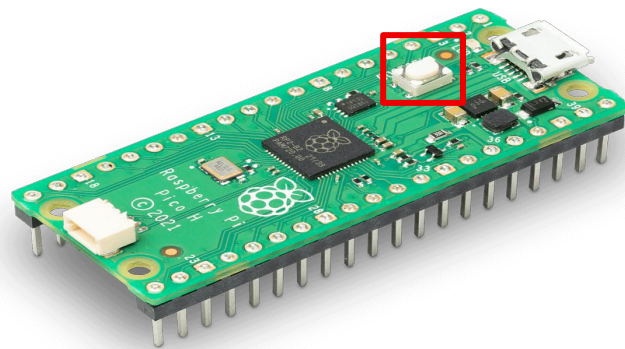
- sshs.uf2
 - File to flash on the device
 - sshs.elf
 - Use to debug and get symbols
 - Snapshot contents
 - Use for your rehosting program
 - Snapshot taken at entry to `assignment_2B_rehost()`
 - Files:
 - `regs.txt`
 - `fw.bin`
 - `rom.bin`
 - `sram.bin`
-

Assignment 2A: First Contact

- Part 2A: Setup
 - Flash the target firmware (sshs.uf2)
 - Download the [Logic 2 software](#)
 - Connect to the Pico via USB (e.g., via minicom/putty)
- Part 2A_UART
 - Decode a UART signal and send a response from the probe
- Part 2A_SPI
 - Find the SPI port and decode the signal

Flashing Firmware

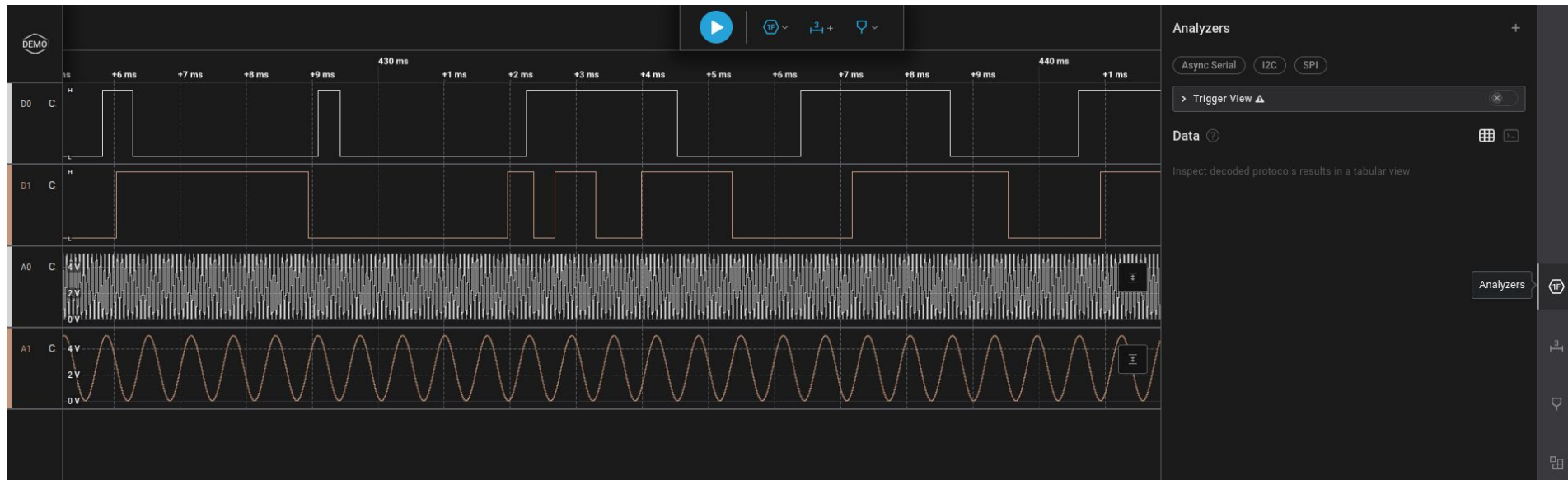
- Option 1: USB Bootloader
 - Press the BOOTSEL button when connecting the pico to USB
 - A flash drive ("RPI-RP2") should appear
 - Copy sshs.uf2 to it
- Option 2: picotool
 - More information in the pico getting started [[guide](#)]



Setup: Expected Result

```
.....  
.....  
.....  
.....  
.....  
.....  
  
..#####..#####.##.....##.#####.  
##...##.##...##.##...##.##...##  
##.....##.....##.....##.##.....  
..#####..#####.#####.#####.  
.....##.....##.##.....##.....##  
##...##.##...##.##...##.##...##  
..#####..#####.##.....##.#####.
```

LOGIC 2/Traces



Assignment 2B: Rehosting

- The pico has a Pin functionality
 - When providing correct pin, flag will be printed
 - However: Pin is unknown!
- Your task: recover the pin using rehosting

The Problem

```
void assignment_2B_rehost (void) {
    uint16_t pin;
    int ret;
    uint8_t flag[sizeof(REHOST_FLAG)];

    printf("What is the pin?");
    scanf("%hd", &pin);
    sleep_ms(100);
    printf("Received PIN %d\n", pin);
    sleep_ms(100);
    puts("Attempting to retrieve flag");

    memcpy(flag, REHOST_FLAG, sizeof(REHOST_FLAG));
    decrypt_rehosting_flag (pin, flag, sizeof(REHOST_FLAG));
    printf("%s\n", flag);
}
```

The Problem

```
void assignment_2B_rehost (void) {
    uint16_t pin;
    int ret;
    uint8_t flag[sizeof(REHOST_FLAG)];

    printf("What is the pin?");
    scanf("%hd", &pin);
    sleep_ms(100);
    printf("Received PIN %d\n", pin);
    sleep_ms(100);
    puts("Attempting to retrieve flag");

    memcpy(flag, REHOST_FLAG, sizeof(REHOST_FLAG));
    decrypt_rehosting_flag (pin, flag, sizeof(REHOST_FLAG));
    printf("%s\n", flag);
}
```



The Solution: Unicorn Engine

- A QEMU Fork
- Added hooks for memory, basic blocks, memory, ...
- CPU Emulator, Fork of QEMU
- Multi-architectures: **Arm**, Arm64 (Armv8), M68K, Mips, Sparc, & X86 (include X86_64) PowerPC, RISC5

Meet: Unicorn



- QEMU-based CPU emulation framework
 - Removed code for memory and peripherals
- Various language bindings:
 - Rust
 - Python
 - ...
- Allows quickly prototyping for rehosting small parts of a firmware



<https://www.unicorn-engine.org>

20



Unicorn Engine

- QEMU plus API minus everything else
 - "just runs" code
 - Easy to set up
 - Bindings in many popular languages
 - Hard to scale:
 - no peripherals
 - no interrupt handling
 - no coprocessors
 - no nothing
-



Note on Unicorn Python Bindings

- Programming Unicorn in python is fun!
 - Quick prototyping
 - Dynamic debugging via:
 - `import IPython; IPython.embed()`
 - Unfortunately, python has many performance footguns
 - E.g, slicing (results into internal copy of data in most cases)
 - Can be problematic for fuzzing, just need to be aware!
 - For max performance, can use other languages
 - For this exercise, we recommend Python
-



Unicorn Emulator APIs

- Important API calls:
 - `uc.mem_map(addr, size, permissions)`
 - `uc.reg_write(reg, value)`
 - `uc.mem_write(addr, content)`
 - `uc.emu_start(start_addr, end_addr)`
 - More information:
 - Main tutorial: <https://www.unicorn-engine.org/docs/tutorial.html>
 - Thumb+Python specific:
https://github.com/unicorn-engine/unicorn/blob/master/bindings/python/tests/test_arm.py
-

API calls #1

```
uc.mem_map(address, size, perms=perms)
```

Creates a memory mapping. Arguments:

- address: base address of new mapping
 - Needs to be page aligned!
- size: size of mapped range
 - Needs to be multiple of page size (0x1000)
- Perms: permissions, can be combined via "|":
 - UC_PROT_NONE
 - UC_PROT_READ
 - UC_PROT_WRITE
 - UC_PROT_EXEC
 - UC_PROT_ALL

```
uc.mem_map(0x40000000, 0x1000, perms=UC_PROT_READ | UC_PROT_WRITE)
```

API calls #2

`uc.mem_read(address, size)`

Reads memory and returns it. Arguments:

- address: address to read from
- size: number of bytes to read

`uc.mem_write(address, content)`

Writes content to address. Arguments:

- address: address to write to
- content: bytes to write

```
val = uc.mem_read(0x40000000, 4)
uc.mem_write(0x40000000, b'\xef\xbe\xad\xde')
```

API calls #3

`uc.reg_read(reg)`

Reads register content and returns it. Arguments:

- `reg`: register to read from. For arm32/thumb, must be one of:
 - `UC_ARM_REG_R0`
 - `UC_ARM_REG_R1`
 - ...
 - `UC_ARM_REG_R12`
 - `UC_ARM_REG_SP`
 - `UC_ARM_REG_LR`
 - `UC_ARM_REG_PC`

`uc.reg_write(reg, content)`

Writes content to register. Arguments:

- `reg`: reg to write to, as above
- `Content`: value to write

```
ret = uc.reg_read(UC_ARM_REG_R0)
uc.reg_write(UC_ARM_REG_R0, 0xdeadbeef)
```

API calls #4

`uc.emu_start(start_addr, end_addr, count=cnt)`

Kicks off emulation via unicorn. Arguments:

- `start_addr`: Where to start emulation
- `end_addr`: Where to stop emulation (if address is hit)
- `cnt`: max number of instructions to execute

Important:

- You need to set the thumb bit in `start_addr` if you want to emulate thumb code

```
uc.emu_start(Reset_Handler | 1, until=ADDR_Reset_Handler_copy_loop_end, count=100000)
```

More information (1)



Unicorn

The Ultimate CPU emulator

[Download](#)

[Docs](#)

[Showcase](#)

[Contact](#)

Unicorn is a lightweight multi-platform, multi-architecture CPU emulator framework.

Highlight features:

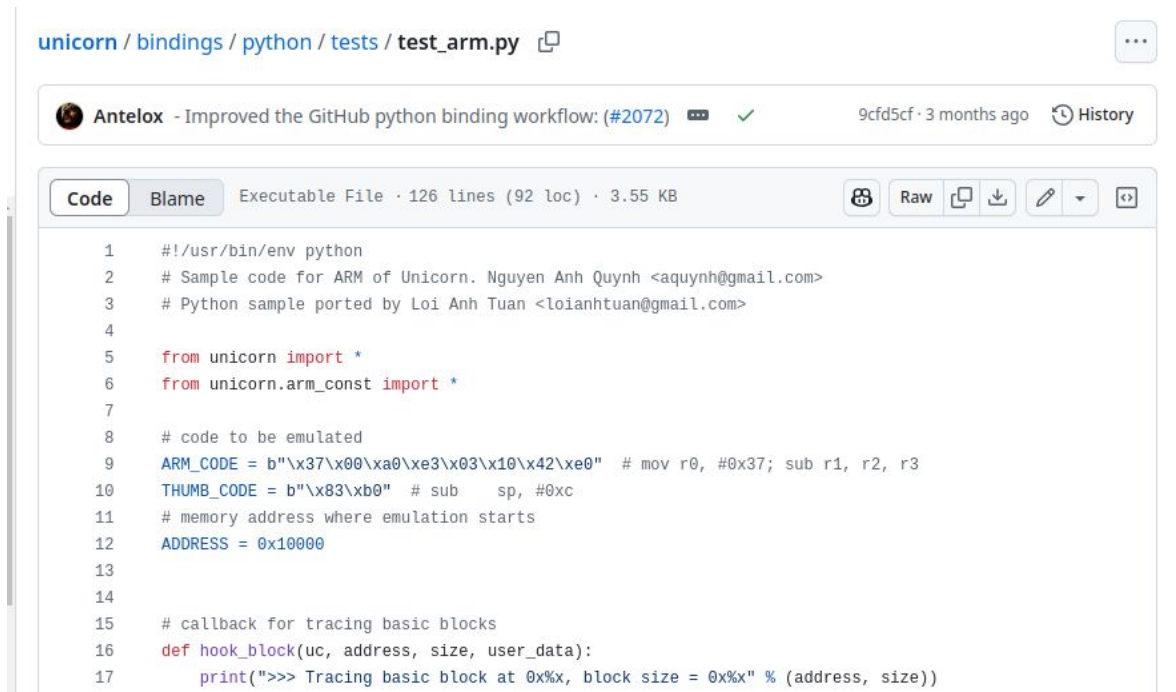
- Multi-architectures: Arm, Arm64 (Armv8), M68K, Mips, Sparc, & X86 (include X86_64).
- Clean/simple/lightweight/intuitive architecture-neutral API.
- Implemented in pure C language, with bindings for Pharo, Crystal, Clojure, Visual Basic, Perl, Rust, Haskell, Ruby, Python, Java, Go, .NET, Delphi/Pascal & MSVC available.
- Native support for Windows & *nix (with Mac OSX, Linux, *BSD & Solaris confirmed).
- High performance by using Just-In-Time compiler technique.
- Support fine-grained instrumentation at various levels.
- Thread-safe by design.
- Distributed under free software license GPLv2.

Find in this [BlackHat USA 2015 slides](#) more technical details behind Unicorn engine.

Unicorn is based on [QEMU](#), but it goes much further with [a lot more to offer](#).



More information (2)



unicorn / bindings / python / tests / test_arm.py

Antelox - Improved the GitHub python binding workflow: (#2072) 9cfd5cf · 3 months ago History

Code Blame Executable File · 126 lines (92 loc) · 3.55 KB

```
1  #!/usr/bin/env python
2  # Sample code for ARM of Unicorn. Nguyen Anh Quynh <aquynh@gmail.com>
3  # Python sample ported by Loi Anh Tuan <loianhtuan@gmail.com>
4
5  from unicorn import *
6  from unicorn.arm_const import *
7
8  # code to be emulated
9  ARM_CODE = b"\x37\x00\xa0\xe3\x03\x10\x42\xe0" # mov r0, #0x37; sub r1, r2, r3
10 THUMB_CODE = b"\x83\xb0" # sub sp, #0xc
11 # memory address where emulation starts
12 ADDRESS = 0x10000
13
14
15 # callback for tracing basic blocks
16 def hook_block(uc, address, size, user_data):
17     print(">>> Tracing basic block at 0x%x, block size = 0x%x" % (address, size))
```

GITHUB !

```

from unicorn import *
from unicorn.arm_const import *

# code to be emulated
ARM_CODE    = b"\x37\x00\xa0\xe3\x03\x10\x42\xe0 " # mov r0, #0x37; sub r1, r2, r3
THUMB_CODE  = b"\x83\xb0" # sub    sp, #0xc
# memory address where emulation starts
ADDRESS     = 0x10000

# callback for tracing basic blocks
def hook_block(uc, address, size, user_data):
    print(">>> Tracing basic block at 0x %x, block size = 0x %x" %(address, size))

# callback for tracing instructions
def hook_code(uc, address, size, user_data):
    print(">>> Tracing instruction at 0x %x, instruction size = 0x %x" %(address, size))

```

```
def test_thumb():  
    print("Emulate THUMB code")  
    try:  
        # Initialize emulator in thumb mode  
        mu = Uc(UC_ARCH_ARM, UC_MODE_THUMB)  
  
        # map 2MB memory for this emulation  
        mu.mem_map(ADDRESS, 2 * 1024 * 1024)  
  
        # write machine code to be emulated to memory  
        mu.mem_write(ADDRESS, THUMB_CODE)  
  
        # initialize machine registers  
        mu.reg_write(UC_ARM_REG_SP, 0x1234)  
  
        [...]
```

```
# tracing all basic blocks with customized callback
mu.hook_add(UC_HOOK_BLOCK, hook_block)

# tracing all instructions with customized callback
mu.hook_add(UC_HOOK_CODE, hook_code)

# emulate machine code in infinite time
# Note we start at ADDRESS | 1 to indicate THUMB mode.
mu.emu_start(ADDRESS | 1, ADDRESS + len(THUMB_CODE))

# now print out some registers
print(">>> Emulation done. Below is the CPU context")

sp = mu.reg_read(UC_ARM_REG_SP)
print(">>> SP = 0x%x" % sp)

except UcError as e:
    print("ERROR: %s" % e)
```

The Plan

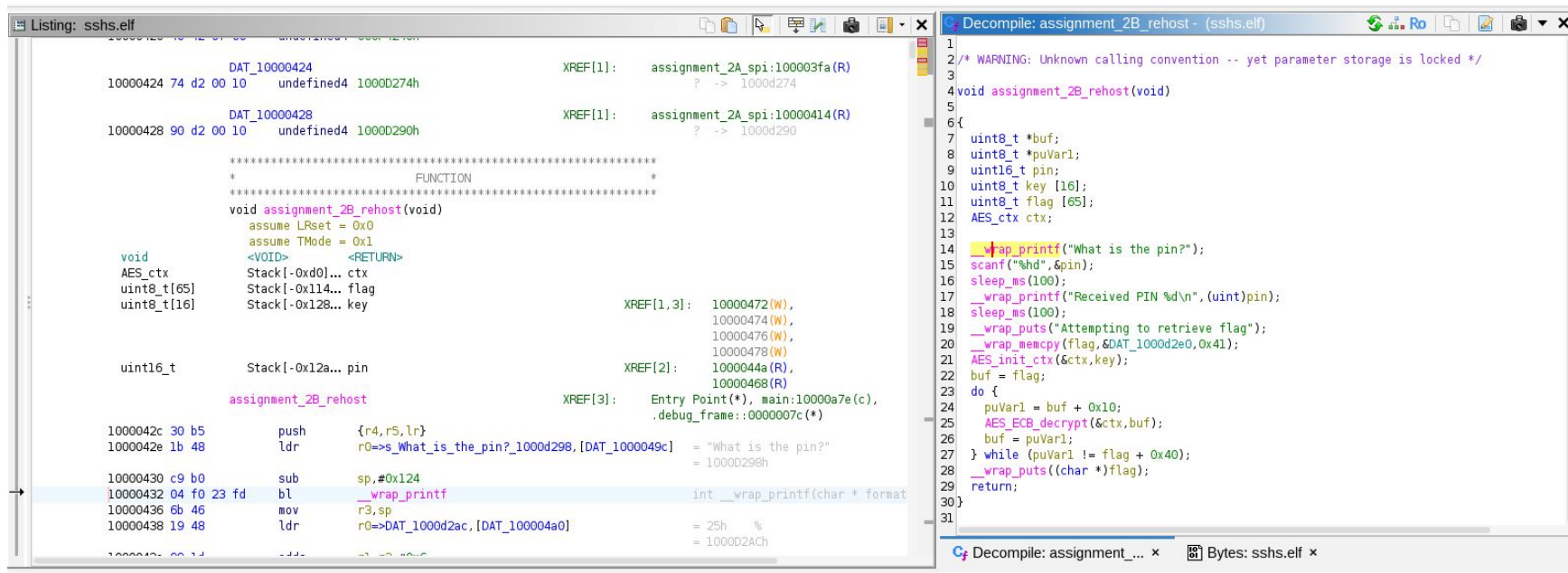
1. **Load the provided snapshot** in unicorn engine (c.f. [test_arm.py](#)).
 2. Create a **hook for all the functions** that must be emulated (e.g. printf)
 3. **Start emulation** to brute force the pin and get the flag!
-

Getting more information: Reverse Engineering

We also provide a sshs.elf file

- You can inspect it in a reverse engineering framework, such as ghidra, binary ninja, or IDA
 - Really helpful to find function addresses, relevant registers, and more!
 - Under normal circumstances, you would not have symbols
- Alternative: objdump:
 - `$ arm-none-eabi-objdump -d sshs.elf`

Ghidra



The screenshot displays the Ghidra IDE interface with two main windows. The left window, titled 'Listing: sshs.elf', shows assembly code for the 'assignment_2B_rehost' function. The right window, titled 'Decompile: assignment_2B_rehost - (sshs.elf)', shows the corresponding decompiled C code.

Assembly Code (Left Window):

```
10000424 74 d2 00 10    undefined4 1000d274h
10000428 90 d2 00 10    undefined4 1000d290h

*****
*               FUNCTION
*               *****
void assignment_2B_rehost(void)
    assume LRset = 0x0
    assume TMode = 0x1
    <VOID>
    <RETURN>
    void
    AES_ctx
    uint8_t[65]
    uint8_t[16]
    Stack[-0xd0]... ctx
    Stack[-0x114]... flag
    Stack[-0x128]... key

    uint16_t
    Stack[-0x12a]... pin

    assignment_2B_rehost

1000042c 30 b5    push    {r4,r5,lr}
1000042e 1b 48    ldr     r0=>s_What_is_the_pin?_1000d298,[DAT_1000049c]
                                     = "What is the pin?"
                                     = 1000d298h

10000430 c9 b0    sub     sp,#0x124
10000432 04 f0 23 fd bl      __wrap_printf
10000436 6b 46    mov     r3,sp
10000438 19 48    ldr     r0=>DAT_1000d2ac,[DAT_100004a0]
                                     = 25h
                                     = 1000d2ACh
```

Decompiled Code (Right Window):

```
1
2 /* WARNING: Unknown calling convention -- yet parameter storage is locked */
3
4 void assignment_2B_rehost(void)
5
6 {
7     uint8_t *buf;
8     uint8_t *puVar1;
9     uint16_t pin;
10    uint8_t key [16];
11    uint8_t flag [65];
12    AES_ctx ctx;
13
14    __wrap_printf("What is the pin?");
15    scanf("%hd",&pin);
16    sleep_ms(100);
17    __wrap_printf("Received PIN %d\n",(uint)pin);
18    sleep_ms(100);
19    __wrap_puts("Attempting to retrieve flag");
20    __wrap_memcpy(flag,&DAT_1000d2e0,0x41);
21    AES_init_ctx(&ctx,key);
22    buf = flag;
23    do {
24        puVar1 = buf + 0x10;
25        AES_ECB_decrypt(&ctx,buf);
26        buf = puVar1;
27    } while (puVar1 != flag + 0x40);
28    __wrap_puts((char *)flag);
29    return;
30 }
31
```

Assignment 2 remarks

- Documentation is your best friend! Have a look the linked datasheets!
- The assignment description on canvas is full of hints and helpful tips.



imgflip.com

JAKE-CLARK.TUMBLR

Plagiarism

- We take plagiarism seriously.
 - We expect that your submissions reflect the work of your group
 - And that the submission is formulated in **your own words**
 - Do not copy code/text from third parties without referencing it
 - This includes submitting generative AI generated texts/code as your own work
 - This includes **also** the use of generative AI based translation tools:
 - *“In particular, generative AI or other editorial assistance must not be used to: [...] Translate text into, or from, the language being studied.”*
- UoB’s Code of Practice on Academic Integrity, A.1.6 [\[link\]](#)

TA Support

If you have questions, please reach out to all three TAs via email:

- Jinjin Wang <jxw1663@student.bham.ac.uk>
- Yusuf Hegazy <yah408@student.bham.ac.uk>
- Make sure to include **all two** in your emails
- And when answering, to use **reply all**

Hardware Handout

- Don't forget to assign yourself to groups (for assignment 2)
 - <https://canvas.bham.ac.uk/courses/83848/groups#tab-32541>
 - Hardware is only issued for groups of 3 people!
- Missed the in-person handout lecture?
 - No problem! Speak to Marius or the TAs in the upcoming sessions (or send an email!)
 - We will have hardware for handout with us
- Return the hardware after the assignment(!)
 - We do keep track of handed out hardware
 - Teams which did not return hardware will not receive marks for this exercise
 - Please return